Shortcut Learning and Its Mitigation in Graph Neural Networks for Cyber-Physical Risk Assessment: A Cross-Dataset Study on ICS/SCADA Attack-Path Detection

Misbah Shaheen

School of Electrical Engineering and Computer Science, National University of Sciences and Technology (NUST), Islamabad, Pakistan

Abstract

Graph neural networks (GNNs) are increasingly proposed for risk assessment in industrial control systems (ICS) and SCADA networks, where communication topology and packet-level features can jointly signal an ongoing attack. A recurring but under-reported failure mode in this setting is *shortcut learning*: a model reaches apparently strong detection metrics by exploiting an artifact of the evaluation protocol rather than a property of the attack itself. This paper documents a complete shortcut-learning investigation carried out across three ICS/SCADA datasets – ICS-Flow, BATADAL, and SCADANet – using a GraphSAGE-based edge classifier. We show that an IP-address-held-out evaluation protocol produces near-perfect recall (1.00) on SCADANet, but that this figure is structurally guaranteed for ten of thirteen attack types because they are generated from a single fixed source IP and are therefore largely absent from any IP-held-out test set; a flow-level, subtype-balanced protocol exposes a 27.2% false-positive rate on held-out normal traffic instead. We find evidence against topology/degree leakage as the cause of these false positives via a constant-feature node ablation, and instead trace approximately 95% of them to two high-volume hosts using host-level forensics, later supported quantitatively with GNNExplainer, PGExplainer, and Integrated Gradients. A leakage-safe threshold-calibration procedure – selecting the decision threshold on a calibration subset carved from the training flows, which never touches test labels – reduces the false-positive rate from 27.2% to 15.2% at a recall cost of 0.0023, without retraining the model. We further show that switching from a static, IP-held-out evaluation to a chronological (temporal) evaluation collapses SCADANet's F1 from 0.993 to 0.629 and BATADAL's F1 from 0.862 to 0.778, and that the two datasets fail in opposite directions (recall collapse vs. precision collapse). A label-permutation test (5 seeds, ~20× real-vs-shuffled F1 gap) and a rolling split-boundary sensitivity check argue against memorization and a lucky split boundary as explanations for BATADAL's temporal recall of 1.00, while a 5-seed stability sweep shows SCADANet's temporal recall collapse (0.51 ± 0.02) is a stable and reproducible property of the model rather than an unlucky initialization. We argue that this arc – suspicious result, competing-hypothesis ablation, host-level root cause, quantitative explainability support, and leakage-safe mitigation – constitutes a stronger and more transferable methodological contribution for cyber-physical risk assessment than the specific GraphSAGE architecture used to obtain it.

Keywords: graph neural networks, shortcut learning, industrial control systems, SCADA, intrusion detection, explainability, cyber-physical systems, GraphSAGE

1. Introduction

Industrial control systems (ICS) and SCADA networks increasingly expose an attack surface that is simultaneously a communication network and a physical process: a malicious flow between two hosts can translate directly into a manipulated sensor reading, an unsafe actuator command, or a disrupted supply of water or power (Taormina et al., 2018). Because attacks in this setting propagate along both cyber (network) and physical (process) edges, graph-structured representations – and graph neural networks (GNNs) in particular – have become an attractive tool for cyber-physical risk

Email address: mshaheen.bsds23seecs@seecs.edu.pk (Misbah Shaheen)

assessment: node and edge features capture packet- or sensor-level content, while the graph topology encodes which devices can influence which other devices (Sun et al., 2024; Wang et al., 2024).

A GNN-based detector that reports near-perfect recall on a held-out test set is, on the surface, a success. In practice, near-perfect metrics on network-security benchmarks are frequently a symptom of *shortcut learning*: the model has found a decision rule that is correlated with the label in the specific evaluation protocol used, but that does not correspond to the semantic property the detector is meant to learn (Geirhos et al., 2020). Shortcut learning is well documented in image and language models, but is comparatively underexamined for graph-structured, node/edge-classification settings on real network-security data, where the shortcut can hide in the evaluation *split* itself (e.g., which IP addresses or which time windows are held out) rather than in an obviously spurious pixel or token.

This paper reports a complete shortcut-learning investigation carried out over an eight-week internship project on graph-based risk assessment for cyber-physical systems, using a GraphSAGE-based edge classifier (Hamilton et al., 2017) evaluated across three ICS/SCADA datasets of different character: ICS-Flow, a small ICSSIM-testbed capture with an explicit knowledge-graph layer; BATADAL, a simulated water-distribution SCADA dataset (Taormina et al., 2018); and SCADANet, a larger labeled packet-capture dataset containing 13 attack types plus normal traffic. Rather than presenting the final architecture as the contribution, we present the investigation itself: an initial near-perfect result that we did not trust, a systematic ablation that found evidence against the most likely confound (topology/degree leakage), a host-level forensic analysis that identified the true cause, a quantitative explainability pass using three independent methods, and a leakage-safe mitigation whose effect is isolated from retraining. We repeat a structurally similar investigation for a second, unrelated shortcut: an IP-held-out evaluation protocol whose resulting test set contains only 2 of 13 SCADANet attack types, and a static (non-temporal) evaluation protocol whose near-perfect metrics do not survive a switch to chronological evaluation.

The contributions of this paper are:

1. We document a full shortcut-learning diagnosis on real ICS/SCADA data: an evaluation-protocol shortcut (an IP-held-out test set covering only 2 of 13 attack types), a content-feature root cause for a 27.2% false-positive rate (finding evidence against topology/degree leakage via a constant-node-feature ablation), and quantitative support for that root cause with GNNExplainer, PGExplainer, and Integrated Gradients.
2. We introduce a leakage-safe threshold-calibration procedure that isolates the effect of decision-threshold choice from retraining, and show it reduces SCADANet's false-positive rate from 27.2% to 15.2% at negligible recall cost (0.9997 $\rightarrow$ 0.9974).
3. We benchmark static vs. temporal (chronological) evaluation across two datasets and show they fail in opposite directions – SCADANet loses 36.4 F1 points via a recall collapse, BATADAL loses 8.4 F1 points via a precision collapse – and use a label-permutation test, a rolling split-boundary check, and a 5-seed stability sweep to distinguish a genuine, reproducible temporal generalization gap from memorization or an unlucky split/seed.

2. Related Work

2.1. GNNs for network and ICS intrusion detection

Graph-based intrusion detection systems represent hosts or flows as graph elements and use message passing to combine topological and content information. E-GraphSAGE (Lo et al., 2022) and GNN-IDS (Sun et al., 2024) are representative examples that apply GraphSAGE-family architectures to network-flow classification, reporting strong aggregate metrics on public intrusion-detection benchmarks. Recent surveys of GNN-based intrusion detection (Wang et al., 2024; Zhong et al., 2024) identify evaluation-protocol design – particularly the choice of what is held out between train and test – as a persistent methodological weak point across the field, echoing concerns raised earlier for classical (non-graph) network IDS benchmarks. Closer to the cyber-physical setting studied here, Sweeten et al. (2023) fuse cyber (network) and physical (sensor) features in a single heterogeneous GNN for smart-grid intrusion detection, and Li and Chasaki (2024) introduce non-adjacent "express edges" to let a heterogeneous GNN capture multi-hop attack propagation that standard message passing misses; both report strong aggregate detection metrics without examining whether an evaluation-protocol artifact – rather than the graph structure itself – is responsible. Koistinen et al. (2026) evaluate an attention-based spatio-temporal GNN on the SWaT ICS testbed and explicitly caution that F1-score maximization can be a misleading model-selection criterion for ICS anomaly detectors: the operating point that maximizes F1 on their

data yields a markedly higher false-positive rate than a conformal-prediction threshold calibrated to a fixed false-alarm budget, and one of their baselines reaches a near-tied F1 score while detecting only 2 of 41 labeled attacks. This concern – that a single aggregate metric can obscure how a strong-looking result was actually achieved – is precisely the failure mode our shortcut-learning investigation targets, though we diagnose it through a different mechanism (evaluation-split structure and content-feature host artifacts, rather than threshold choice alone). Our work targets exactly this weak point on ICS/SCADA data specifically, rather than on the general-purpose intrusion-detection benchmarks (e.g. CICIDS, NSL-KDD) that dominate this literature.

2.2. Shortcut learning in deep and graph models

Geirhos et al. (2020) formalize shortcut learning as the use of decision rules that solve the training distribution without solving the intended task, and argue that shortcuts are best diagnosed by testing generalization under a deliberately shifted evaluation condition rather than by inspecting in-distribution accuracy alone. Follow-up work has shown that shortcuts can originate in early training dynamics (Murali et al., 2023) and has proposed foundational accounts of why gradient-based learning is biased toward simple, spuriously predictive features (Hermann et al., 2023). In the GNN literature specifically, node- or edge-degree can act as an unintended shortcut when node features are weak or absent, because message passing implicitly encodes degree information through neighborhood aggregation; this is the concern that motivates our topology-leakage ablation in Section 3 rather than assuming a content-feature explanation a priori.

2.3. ICS/SCADA and cyber-physical datasets

ICS-Flow is generated from the ICSSIM testbed and provides labeled network flows alongside PLC-level context, making it suitable for building an explicit knowledge-graph layer over a small number of devices. BATADAL, released for the "Battle of the Attack Detection Algorithms" competition on the C-Town water-distribution network, provides simulated SCADA sensor readings (tank levels, pump flows and statuses, junction pressures) at hourly resolution and is widely used as an anomaly-detection benchmark for water infrastructure (Taormina et al., 2018; Conti et al., 2021); across the full competition's training and test releases the challenge totals 14 labeled cyber-physical attacks, of which the specific file used in this study (the unlabeled test release, subsequently disclosed) contains 7. SCADANet is a larger labeled packet-capture dataset (534,841 packets, 60 raw columns, 13 attack classes plus normal traffic) that, unlike BATADAL, exposes raw network-flow content alongside topology, which is what enables the host-level explainability analysis in Section 4. Existing use of BATADAL in the anomaly-detection literature has focused on classical time-series and autoencoder methods (Somma, 2025); we are not aware of prior work that benchmarks the same GNN architecture across ICS-Flow, BATADAL, and SCADANet under both static and temporal evaluation protocols, which is the comparison this paper contributes.

2.4. Explainability for graph neural networks

GNNExplainer (Ying et al., 2019) and PGExplainer (Luo et al., 2020) produce post-hoc soft masks over edges and node features that approximate which parts of a local subgraph were responsible for a given prediction; the two methods differ in that GNNExplainer optimizes a mask per instance while PGExplainer amortizes mask prediction across instances via a trained explainer network, which is why we treat their outputs as complementary evidence rather than directly comparable numbers. Integrated Gradients (Sundararajan et al., 2017), as implemented in the Captum library (Kokhlikyan et al., 2020), instead attributes a prediction to individual input features by integrating gradients along a straight-line path from a baseline input, which is appropriate for our edge-attribute (packet-content) feature vector but answers a different question from an edge mask and is on a different numeric scale.

Beyond explainability, temporal graph-learning methods such as Euler (King and Huang, 2023) show that link-prediction-style GNNs can detect lateral movement from scalable temporal snapshots, motivating our own streaming-topology construction for temporal evaluation in Section 3; degree- and sample-size-sensitivity issues in small-traffic GNN settings (Yehezkel and Elyashiv, 2022) and efficient edge/feature representations for GNN-IDS (Friji et al., 2023; Tran and Park, 2024) further motivate our topology-ablation methodology. We also note that GNN-based detectors are not immune to adversarial evasion (Yan et al., 2024). Physics-informed modeling of the underlying physical process is a natural complement to the network-layer detection studied here, which is the direction we point toward for RQ3 (physical-process impact modeling) in Section 5.

3. Proposed Method

3.1. Datasets and knowledge-graph construction

We use three datasets that differ in scale and character, summarized in Table 1. ICS-Flow (Dehlaghi-Ghadim et al., 2023) (44,156 labeled flows from an ICSSIM testbed) is used to build an explicit multi-relational knowledge graph, in the style of security knowledge graphs such as SEPSES (Kiesling et al., 2019), with four node types – `Asset`, `Attacker`, `Technique`, and `CVE` – the latter two forming a threat-intelligence overlay for MITRE ATT&CK ICS techniques (MITRE, 2024) and CVEs, connected via `COMMUNICATES_WITH`, `USES_TECHNIQUE`, `TARGETS`, `VULNERABLE_TO`, `TRIGGERED_ON`, and `ENABLES` edges (7,685 nodes, 15,402 edges in total; Table 2). This knowledge graph is exported to Neo4j for ranked attack-path querying via Cypher. BATADAL provides 4,177 hourly SCADA readings across 43 sensor and actuator variables (7 tank levels, 11 pump flow rates, 11 pump statuses, 1 valve flow rate, 1 valve status, and 12 junction pressures) with a corrected binary attack flag (5.2% attack rate; 7 attack events in the processed file). SCADANet provides 534,841 raw packets across 60 columns and 13 attack classes plus normal traffic (84.3% attack traffic in the raw capture); a column audit removes 9 dead/constant, 2 leaking, and 24 identifier/free-text columns, leaving 10 numeric and 13 categorical raw columns that expand to 224 one-hot/numeric feature dimensions after encoding.

3.2. Model

We use two-layer GraphSAGE-based classifiers (Hamilton et al., 2017) across the three datasets: an edge-level classifier (`EdgeClassifierWithAttr`) for ICS-Flow and SCADANet, and a graph-level window classifier for BATADAL. For the edge-level model, a `SAGEConv`-based embedder produces a 16-dimensional node embedding via mean-aggregation message passing, and an edge/flow-level prediction is formed by concatenating the embeddings of the two endpoint nodes with an edge-attribute vector. For SCADANet's 224-dimensional one-hot edge-attribute vector, a linear `edge_proj` layer projects it to 16 dimensions before concatenation, keeping the classifier head's input size comparable across the enriched and baseline variants. BATADAL is instead evaluated with a graph-level `WindowGraphClassifier` that pools node embeddings over each sliding-window graph. Class imbalance is handled throughout via inverse-frequency class weighting rather than resampling; for SCADANet's Track B protocol (Section 3.3), weighting is applied per attack subtype rather than by binary label alone, so that rare attack types receive training signal comparable to the dominant `udp_flood` class, and reported metrics reflect the natural class distribution of each test set. Model sizes range from 706 parameters (BATADAL) to 5,090 parameters (SCADANet enriched), with full-batch inference latency ranging from 1.97 ms (BATADAL) to 289.33 ms (SCADANet enriched) on a single GPU (Table 3).

3.3. Evaluation protocols

We deliberately evaluate SCADANet under two protocols that test different properties. **Track A (IP-held-out)** withholds 20% of IP addresses and evaluates flows touching a held-out IP, testing generalization to previously unseen attacker infrastructure. **Track B (flow-level, subtype-balanced)** instead holds out flows while allowing the same attacker IP to appear in both train and test, testing recognition of known attacker behavior. Track A is intended to evaluate generalization to previously unseen source IP addresses. However, the resulting held-out test population contains only two of SCADANet's 13 attack types: `tcp_syn_flood` and `udp_flood`. The remaining 11 attack types are structurally absent from the Track A test set because their source-IP diversity is insufficient to produce held-out examples under the split construction. Consequently, Track A's headline metrics are computed over a test set of n=10,940 flows and should be interpreted specifically as performance on the subset of attack types for which unseen-IP evaluation is possible, rather than as performance across the full SCADANet attack taxonomy. Track B is used for all diagnostic and mitigation work in Section 4.

For temporal evaluation, SCADANet flows are sorted chronologically and divided into 20 equal-*count* (not equal-*time*) windows, because the raw capture is bursty and equal-time windowing concentrates almost all flows into a handful of windows. A streaming topology is built by accumulating observed IP-pair edges window-by-window, and a 70/30 chronological split trains on the union of the first 14 windows' topology and evaluates on the remainder. BATADAL's temporal evaluation instead uses 24-hour sliding windows with a 6-hour stride (693 total windows, 5-fold cross-validation for the static protocol; a chronological 70/30 split for the temporal protocol, with a small window overlap of 3 windows reported explicitly at the boundary rather than hidden).

Table 1: Dataset summary.

	ICS-Flow	BATADAL	SCADANet
Unit of analysis	flow	hourly reading	packet/flow
Flows used (post-cleaning)	44,156 flows	4,177 rows	534,841 packets
Attack classes	binary	binary attack flag	13 attack classes + normal
Attack rate	17.0% (edge test set)	5.2%	84.3% (raw)
Graph unit	9 IP nodes (8 assets + 1 multicast)	24h/6h sliding window	6,414 IP nodes

Table 2: ICS-Flow knowledge graph: node and edge inventory.

Node type	Count
Asset	6
Attacker	2
Technique (MITRE ATT&CK ICS)	4
CVE	4
Edge type	**Count**
`COMMUNICATES_WITH`	23
`USES_TECHNIQUE`	15
`TARGETS`	10
`VULNERABLE_TO`	11
`TRIGGERED_ON`	15,338
`ENABLES`	5

3.4. Topology-shortcut ablation and threshold-calibration methodology

To investigate topology/degree leakage as a possible source of Track B's false positives, we ablate the node-embedding pathway directly: SCADANet node features are constant placeholder vectors by design (all nodes start from an identical input), so if message passing were injecting a degree-correlated shortcut into node embeddings, embeddings across nodes would either be identical (indicating literally zero signal reaches the embedder) or show a rate of variation traceable to node degree. We report the standard deviation of node embeddings across nodes and an explicit identity check (`torch.allclose`) to distinguish "the embedder is not learning anything useful" from "the embedder is not leaking degree."

For threshold calibration, we deliberately fix the trained model and vary only the decision threshold, to isolate the threshold's effect from any change that would result from retraining on a different split. A calibration subset (15% of the training flows, not the test flows) is carved out via a Generator seeded independently of the train/test split; precision-recall trade-offs are computed on this subset only, and the resulting threshold is applied once to the already-computed test-set probabilities. No test label is used at any point during threshold selection. For BATADAL's temporal split, we extend this to a 5-fold leave-one-fold-out procedure on the training windows only, producing out-of-fold (OOF) probabilities that are never generated by a model that has seen the corresponding label, and selecting both a max-F1 threshold and a precision-floor ($\geq$0.80) threshold from the OOF precision-recall curve before applying either once to the fixed test-set predictions.

4. Evaluation

4.1. Cross-dataset static evaluation

Table 4 reports static evaluation across all three datasets. ICS-Flow's edge classifier reaches an F1 of 0.572 on a real, non-degenerate 80/20 flow split (35,317 train / 8,829 test edges), with precision (0.938) substantially exceeding recall (0.411); the node-level classifier reaches perfect training accuracy on only 8 devices, which we do not report as a generalization result because 8 nodes is too few to split into a meaningful train/test partition. BATADAL's 5-fold windowed cross-validation reaches F1=0.862 (precision 0.790 $\pm$ 0.116, recall 0.965 $\pm$ 0.043). SCADANet's static (IP-held-out, Track A) protocol reaches F1=0.993, but Ta-

Table 3: Model size and full-batch inference latency.

Model	Parameters	Size (MB)	Latency (ms, mean ± std, 50 runs)
ICS-Flow `EdgeClassifier`	1,298	0.0093	5.70 ± 0.36
BATADAL `WindowGraphClassifier`	706	0.0063	1.97 ± 0.18
SCADANet baseline	1,234	0.0093	77.37 ± 10.32
SCADANet enriched	5,090	0.0246	289.33 ± 36.43

ble 5 shows this figure is carried almost entirely by two attack types (`udp_flood`, n=14; `tcp_syn_flood`, n=1,263) with recall of 1.000 each; the remaining 11 attack types are absent from this test set by construction.

4.2. Track A vs. Track B: the evaluation-protocol shortcut

Table 5 contrasts Track A and Track B. Track A's near-perfect metrics reflect only 2 of 13 attack types; Track B, evaluated over a test set that includes 12 of the 13 attack types (n=105,711), reaches F1=0.975 but with a markedly higher false-positive rate on held-out normal traffic (27.2%, precision 0.951) than Track A's 0.2%. Thus, the higher F1 of Track A should not be interpreted as evidence of superior attack detection across the full taxonomy: the two protocols evaluate substantially different attack populations. This gap reflects the different evaluation populations induced by the two protocols: an IP-held-out split is structurally meaningless for the ten attack types generated from a single fixed source IP, because that IP is either entirely in train or entirely in test, so IP-held-out recall for those types is undefined rather than measured; `os_scan` (2 source IPs) falls just short of the diversity threshold used here and is likewise absent from the Track A test set. Track B is therefore the only protocol of the two that is a fair test of the model's ability to recognize known attacker behavior across the full attack taxonomy, and it is Track B's 27.2% false-positive rate that motivates the root-cause investigation in Section 4.3.

4.3. False-positive root cause: investigating topology, identifying content features

The node-embedding ablation described in Section 3.4 shows a small but non-zero standard deviation across node embeddings (0.056) and confirms embeddings are not identical across nodes (`torch.allclose` = False), which is consistent with the model receiving essentially no usable topology/degree signal (constant input features, mean-aggregation message passing) rather than with an implicit degree-leakage shortcut. This provides evidence against the most likely graph-specific confound and redirects the investigation toward the flows' content features.

Host-level forensics show that 4,594 false positives occur on held-out normal traffic, of which 4,374 (95.2%) concentrate on exactly two high-volume hosts, 192.168.119.140 and 192.168.119.141 – the same two hosts that dominate the dataset's attack traffic across nearly every attack type (Table 6). Comparing content-feature means between false positives and true negatives on normal traffic from these hosts shows a large gap on `Tcp_flags_reset_Set` (44.1% of false positives vs. 0.1% of true negatives) and smaller but consistent shifts in `ip_ttl` and `frame_len`, consistent with congestion artifacts on hosts that are concurrently generating or receiving attack traffic.

This qualitative diagnosis receives quantitative support in a subsequent explainability pass over a seeded sample of 40 false-positive flows (of 4,374 total) drawn from the two high-volume hosts. Captum Integrated Gradients (Sundararajan et al., 2017) identifies `Protocol_TCP` (mean |attribution| 0.273), `Tcp_flags_reset_Set` (0.154), and `frame_len` (0.098) as the three strongest content-feature drivers of the attack prediction on these flows, directly matching the host-level forensic finding. GNNExplainer (Ying et al., 2019) produces a low and stable topology-edge-mask maximum (mean 0.292, std 0.002) with a low node-mask saturation fraction (1.1%), indicating a sparse, converged explanation rather than an optimization failure; PGExplainer (Luo et al., 2020) independently produces an even lower edge-mask maximum (mean 0.125, std 0.043). Because topology-explainer edge masks and Integrated Gradients attributions measure different quantities on different numeric scales, we do not compare them directly; instead, all three methods are read as complementary evidence pointing toward the same conclusion – these false positives are driven by packet-content similarity between attack and normal traffic on two specific, high-volume hosts, rather than by graph-topology leakage.

Table 4: Cross-dataset static evaluation.

Dataset (protocol)	Accuracy	Precision	Recall	F1
ICS-Flow (edge, 80/20 split)	0.895	0.938	0.411	0.572
BATADAL (window, 5-fold CV)	0.974 ± 0.012	0.790 ± 0.116	0.965 ± 0.043	0.862 ± 0.054
SCADANet baseline (Track A, 4 dims)	0.998	0.985	1.000	0.992
SCADANet enriched (Track A, 224 dims)	0.998	0.985	1.000	0.993

Table 5: SCADANet Track A (IP-held-out) vs. Track B (flow-level, subtype-balanced).

Metric	Track A	Track B
Accuracy	0.998	0.956
Precision	0.985	0.951
Recall	1.000	1.000
F1	0.993	0.975
Test edges	10,940	105,711
Attack types covered	2 of 13	12 of 13
FPR on normal traffic	0.2%	27.2%

4.4. Leakage-safe threshold calibration

Table 7 compares Track B's default (0.50) decision threshold against a threshold selected on a 15% calibration split carved from the training flows, applied once to the fixed model's existing test-set probabilities. Calibration reduces the false-positive rate from 27.2% to 15.2% and raises precision from 0.951 to 0.972, at a recall cost of only 0.0023 (0.9997 → 0.9974). Because both rows in Table 7 come from the same trained model and the same test-set probabilities, the improvement is attributable entirely to the choice of decision threshold and is not confounded with any change in the learned representation. We report this as a mitigation, not a root-cause fix: the underlying content-feature similarity between attack and normal traffic on the two high-volume hosts remains unaddressed, and per-host feature normalization is identified as future work (Section 5).

4.5. Static vs. temporal evaluation

Table 8 contrasts static and temporal (chronological) evaluation for SCADANet and BATADAL. Temporal evaluation hurts SCADANet far more than BATADAL: SCADANet loses 36.4 F1 points (0.993 → 0.629) while BATADAL loses 8.4 points (0.862 → 0.778). Critically, the two datasets fail in *opposite* directions. SCADANet's temporal recall collapses from 1.00 to 0.49 while precision remains comparatively high (0.87); BATADAL's temporal recall remains at 1.00 while precision falls from 0.79 to 0.64. For BATADAL, this means the temporal model still catches every attack window but at the cost of more false alarms; for SCADANet, the temporal model becomes conservative and silently misses roughly half of all attacks.

SCADANet's temporal recall collapse is concentrated almost entirely in a single attack type. `vuln_scan` (n=37,777 test flows) drops to a recall of 0.008 and alone accounts for 88.6% of all false negatives in the temporal test set, while `icmp_flood`, `http_flood`, `tcp_ack_flood`, `sql_injection`, and `apt_exfil` all retain recall ≥0.95 under the same split. This is not explained by unseen attack classes: every attack type present in the temporal test set was also present during training. Instead, `vuln_scan`'s base rate shifts by 28.5× between the two periods (0.83% of training flows vs. 23.5% of test flows; Table 9), and the streaming topology available at the training boundary contains only 820 of the eventual 6,535 cumulative IP-pair edges – i.e., the model is trained on a short initial burst (50 seconds of wall-clock time) and evaluated on almost the entire remainder of a 2,049-second capture (1,999 seconds, ~40× longer).

4.6. Distinguishing a real generalization gap from memorization or a lucky split

Two concerns naturally arise from Table 8: is BATADAL's temporal recall of 1.00 a real signal or an artifact of memorization or a favorable split boundary, and is SCADANet's temporal recall collapse a stable property of the model or an unlucky training run? We address both with diagnostic checks rather than by retraining until the numbers looked better.

Table 6: False positives on held-out normal traffic by source host.

Source IP	False positives
192.168.119.140	2,347
192.168.119.141	2,027
192.168.119.142	115
192.168.119.143	99

Table 7: Track B decision-threshold calibration (same model, same test-set probabilities).

Threshold	Precision	Recall	F1	FPR (normal)
0.50 (default)	0.951	0.9997	0.975	27.2%
0.79 (calibration-set selected)	0.972	0.9974	0.985	15.2%

For BATADAL, a label-permutation test retrains the identical model and split boundary with `window_labels` randomly shuffled across 5 seeds. Real-label F1 (0.778) is approximately 20× the mean shuffled-label F1 (0.038 ± 0.055), which is inconsistent with the model simply memorizing a label-independent artifact of the split. A rolling split-boundary sensitivity check re-runs the same evaluation at 60/40, 70/30, and 80/20 chronological splits; recall remains at 1.00 across all three boundaries (F1 ranging 0.721–0.793), which argues against the 70/30 boundary having been a lucky cut. A precision-recall curve over the fixed test-set probabilities yields an average precision of 0.881. A subsequent 5-fold leave-one-fold-out threshold calibration on the training windows only, using out-of-fold probabilities and never touching test labels, selects a max-F1 threshold of 0.968. Applied once to the fixed test-set predictions, this improves F1 from 0.778 to 0.808 (+3.0 points), with precision increasing from 0.636 to 0.677 while recall remains 1.000. A precision-floor (≥0.80) operating point selects a threshold of 0.667 but produces the same test-set outcome as the default threshold (precision 0.636, recall 1.000, F1 0.778), indicating that no test-set prediction falls between the two thresholds.

For SCADANet, we retrain the temporal-split model under 5 fixed seeds (42, 0, 1, 7, 123) with class weighting, learning rate, and epoch count otherwise unchanged. Recall varies only within a narrow band (0.495–0.533, mean 0.510 ± 0.018) and F1 within 0.629–0.662 (mean 0.643 ± 0.016). The originally reported, unseeded Week 10 result (recall 0.492, F1 0.628) falls just below this 5-seed range – the lowest recall and lowest F1 observed across the 6 total runs – supporting the conclusion that the temporal recall degradation is stable across random initializations rather than being an artifact of a single unfavorable seed.

We additionally test whether SCADANet's recall collapse on `vuln_scan` specifically is explained by distributional drift on the three content features Section 4.3 identified as the model's strongest predictors. Restricting to `vuln_scan` flows only, `Protocol_TCP`'s rate shifts from 0.982 (train) to 0.988 (test); a two-proportion z-test finds this statistically significant ($z = -3.03$, $p = 0.0025$) but the absolute shift is only 0.6 percentage points. `Tcp_flags_reset_Set` shows no significant shift ($z = -0.23$, $p = 0.82$; both periods ≈0.1%). `frame_len` shows no meaningful shift by either a Kolmogorov–Smirnov test (statistic 0.006, $p = 0.9999$) or by comparing means (63.4 vs. 62.2 bytes) and medians (60.0 bytes, identical). We therefore find no evidence that substantial drift in the three strongest identified content features explains `vuln_scan`'s temporal recall collapse; the 28.5× base-rate shift and the largely unobserved topology at the training boundary remain the better-supported explanations, although we do not have a controlled experiment that isolates base-rate shift from topology incompleteness and therefore report this as an open question rather than a settled conclusion.

5. Discussion

The single result most likely to be quoted from this project in isolation – SCADANet static Track A F1 of 0.993 – is also the single result this paper argues should not be quoted in isolation. It is a correct computation of a specific, narrow evaluation protocol (generalization to unseen attacker IP addresses) that happens to be measurable for only 2 of 13 attack types in this dataset, and it says nothing about the model's ability to recognize known attacker behavior across the other 11, nor about its ability to generalize across time. Track B,

Table 8: Static vs. temporal evaluation.

Dataset	Protocol	Precision	Recall	F1
SCADANet	Static (IP-held-out)	0.985	1.000	0.993
SCADANet	Temporal (70/30 chronological)	0.868	0.493	0.629
BATADAL	Static (5-fold CV)	0.790	0.965	0.862
BATADAL	Temporal (70/30 chronological)	0.636	1.000	0.778

Table 9: `vuln_scan` base-rate shift, SCADANet temporal split.

	Train period	Test period
`vuln_scan` flows	3,093	37,777
Total flows	374,389	160,452
Rate	0.83%	23.54%

threshold calibration, and the temporal benchmark each successively remove a layer of apparent success and replace it with a narrower but better-supported claim: the model detects known attacker behavior at F1=0.975 (Track B) with a false-positive rate that can be halved through leakage-safe calibration (27.2%→15.2%) without retraining; and the same model's temporal generalization is markedly worse than its static generalization, with SCADANet failing via a concentrated recall collapse on a single, low-base-rate-at-train-time attack type, and BATADAL failing via a smaller but non-trivial precision cost.

A limitation shared by both temporal protocols is that neither performs strict online, flow-by-flow inference: SCADANet's temporal evaluation uses the final cumulative test-period topology for all test flows (so later test-period edges are technically available to earlier test examples), and BATADAL's 24-hour/6-hour sliding windows overlap by 18 hours, producing a small number of windows (3, at the 70/30 boundary used here) that share raw rows across the train/test boundary. Both choices are reported explicitly rather than hidden, and neither invalidates the qualitative static-vs-temporal comparison, but a stricter, leakage-free streaming evaluation is left as future work. A second limitation is that the host-level false-positive mitigation in Section 4.3 is a threshold adjustment, not a fix to the underlying cause; per-host feature normalization – z-scoring numeric content features against each source IP's own baseline rather than a global distribution – is a natural next step that directly targets the congestion-artifact hypothesis but was not evaluated in this work. Finally, our explainability sample (40 of 4,374 relevant false positives) and multi-seed sweep (5 seeds) are both modest in scale for reasons of compute budget within an 8-week project; scaling both is a low-risk extension that would tighten the confidence intervals reported here without requiring new methodology.

Beyond ICS-Flow, BATADAL, and SCADANet, we expect the underlying pattern to generalize to other cyber-physical intrusion-detection settings where (i) a small number of source addresses generate the large majority of both attack and legitimate high-volume traffic, and (ii) evaluation splits are constructed along an axis – address, time, or session – that is only valid for a subset of the labeled attack taxonomy. We would therefore recommend, as a general practice for GNN-based cyber-physical risk assessment, reporting per-attack-type coverage of the evaluation split alongside aggregate metrics, and treating any near-perfect recall figure as a prompt for a topology-ablation and host-level forensic pass rather than as a terminal result.

6. Conclusion

We presented a complete shortcut-learning investigation for GNN-based cyber-physical risk assessment, spanning three ICS/SCADA datasets and a common GraphSAGE-based edge-classification architecture. Two distinct evaluation failures were identified and diagnosed: an IP-held-out evaluation protocol whose near-perfect metrics are structurally limited to 2 of 13 SCADANet attack types, and a static (non-temporal) evaluation protocol whose F1 does not survive a switch to chronological evaluation (SCADANet: −36.4 points; BATADAL: −8.4 points, in opposite failure directions). For the first shortcut, a topology-leakage ablation provided evidence against the most likely graph-specific confound, host-level forensics identified the

likely content-feature cause, three independent explainability methods gave quantitative support for it, and a leakage-safe threshold-calibration procedure reduced the resulting false-positive rate by 44% relative (27.2%→15.2%) at negligible recall cost, without retraining. For the second shortcut, a label-permutation test, a split-boundary sensitivity check, and a 5-seed stability sweep together distinguish a genuine, reproducible temporal generalization gap from memorization, a lucky split, or an unlucky seed. We argue that this diagnostic methodology – treat a near-perfect result as a hypothesis to be stress-tested, not a conclusion – is the more transferable contribution of this work, and recommend it as standard practice for future GNN-based cyber-physical intrusion-detection research. Future work includes physical-process impact modeling (propagating a detected network-layer attack to a physical-process risk score), per-host feature normalization to address the root cause identified in Section 4.3, and a strict, leakage-free streaming evaluation protocol for both datasets.

Acknowledgments

This work was carried out as part of an 8-week AI Security internship hosted by CNIT / PNTLab Pisa (TeCIP Institute, Scuola Superiore Sant'Anna), under Project 12: "Graph-Based Risk Assessment for Cyber-Physical Systems."

References

Conti, M., Donadel, D., Turrin, F., 2021. A survey on industrial control system testbeds and datasets for security research. IEEE Communications Surveys & Tutorials 23(4), 2248–2294.

Dehlaghi-Ghadim, A., Helali Moghadam, M., Balador, A., Hansson, H., 2023. Anomaly detection dataset for industrial control systems. IEEE Access 11, 107982–107996.

Friji, H., Olivereau, A., Sarkiss, M., 2023. Efficient network representation for GNN-based intrusion detection. In: International Conference on Applied Cryptography and Network Security. Springer, pp. 532–554.

Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., Wichmann, F. A., 2020. Shortcut learning in deep neural networks. Nature Machine Intelligence 2(11), 665–673.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. Advances in Neural Information Processing Systems 30.

Hermann, K., Mobahi, H., Fel, T., Mozer, M. C., 2023. On the foundations of shortcut learning. arXiv preprint arXiv:2310.16228.

Kiesling, E., Ekelhart, A., Kurniawan, K., et al., 2019. The SEPSES knowledge graph: An integrated resource for cybersecurity. In: International Semantic Web Conference (ISWC). Springer, pp. 198–214.

King, I. J., Huang, H. H., 2023. Euler: Detecting network lateral movement via scalable temporal link prediction. ACM Transactions on Privacy and Security 26(3), 1–36.

Koistinen, K., Hellsten, K., Herttuainen, J., Kaski, K. K., 2026. Spatio-temporal attention graph neural network: Explaining causalities with attention. IEEE Access (advance online publication), doi:10.1109/ACCESS.2026.3702352; arXiv:2603.10676.

Kokhlikyan, N., Miglani, V., Martin, M., et al., 2020. Captum: A unified and generic model interpretability library for PyTorch. arXiv preprint arXiv:2009.07896.

Li, H., Chasaki, D., 2024. Heterogeneous GNN with express edges for intrusion detection in cyber-physical systems. In: 2024 International Conference on Computing, Networking and Communications (ICNC). IEEE, pp. 523–529.

Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., Portmann, M., 2022. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. In: NOMS 2022–2022 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–9.

Luo, D., Cheng, W., Xu, D., Yu, W., Zong, B., Chen, H., Zhang, X., 2020. Parameterized explainer for graph neural network. Advances in Neural Information Processing Systems 33, 19620–19631.

MITRE, 2024. ATT&CK for Industrial Control Systems. `https://attack.mitre.org/matrices/ics/`.

Murali, N., Puli, A., Yu, K., Ranganath, R., Batmanghelich, K., 2023. Shortcut learning through the lens of early training dynamics. arXiv preprint arXiv:2302.09344.

Somma, M., 2025. Hybrid temporal differential consistency autoencoder for efficient and sustainable anomaly detection in cyber-physical systems. arXiv preprint arXiv:2504.06320.

Sun, Z., Teixeira, A. M. H., Toor, S., 2024. GNN-IDS: Graph neural network based intrusion detection system. In: Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES). ACM.

Sundararajan, M., Taly, A., Yan, Q., 2017. Axiomatic attribution for deep networks. In: Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 3319–3328.

Sweeten, J., Takiddin, A., Ismail, M., Refaat, S. S., Atat, R., 2023. Cyber-physical GNN-based intrusion detection in smart power grids. In: 2023 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm). IEEE, pp. 1–6.

Taormina, R., Galelli, S., Tippenhauer, N. O., Salomons, E., Ostfeld, A., et al., 2018. Battle of the attack detection algorithms: Disclosing cyber attacks on water distribution networks. Journal of Water Resources Planning and Management 144(8), 04018048.

Tran, D.-H., Park, M., 2024. FN-GNN: A novel graph embedding approach for enhancing graph neural networks in network intrusion detection systems. Applied Sciences 14(16), 6932.

Wang, X., Wang, X., He, M., Zhang, M., Lu, Z., 2024. Spatial-temporal graph model based on attention mechanism for anomalous IoT intrusion detection. IEEE Transactions on Industrial Informatics 20(3), 3497–3509.

Yan, H., Li, X., Zhang, W., Wang, R., Li, H., Zhao, X., Li, F., Lin, X., 2024. Automatic evasion of machine learning-based network intrusion detection systems. IEEE Transactions on Dependable and Secure Computing 21(1), 153–167.

Yehezkel, A., Elyashiv, E., 2022. A GNN-based approach for detecting network anomalies from small traffic samples. In: 2022 IEEE International Conference on Big Data. IEEE, pp. 6838–6840.

Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J., 2019. GNNExplainer: Generating explanations for graph neural networks. Advances in Neural Information Processing Systems 32.

Zhong, M., Lin, M., Zhang, C., Xu, Z., 2024. A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. Computers & Security 141, 103821.